

Class 15: Feature Engineering: Cleaning, Curation, Selection, Construction

Data-driven Systems Engineering

Agenda

- Why feature engineering is still necessary once data is already encoded.
- Data cleaning: missingness, duplicates, and typed filling.
- Data curation: documenting and validating datasets for reuse.
- Feature selection: filter, wrapper, and embedded methods.
- Feature construction from domain knowledge, for tables and for streams.
- Leakage and availability checks before anything ships.

Learning goals

- Distinguish cleaning, curation, selection, and construction as separate engineering tasks.
- Apply filter, wrapper, and embedded feature selection appropriately.
- Construct new features from domain logic rather than generic transformations.
- Recognize leakage and feature-availability problems before they reach production.

Course Map and Running Cases



Today in the course

Focus: Cleaning, curating, selecting, and constructing features once variables are already numeric.

Bridge: Class 14 gave us a numeric representation of every variable. This class decides which of those variables to keep, fix, or build into something more useful.

Fraud case

In the fraud case, feature work means fixing data-quality issues, keeping only informative variables, and building indicators such as spending deviation from a rolling baseline.

Pasteurization case

In the pasteurization case, feature work means validating sensor readings and constructing rate-of-change and state-aware variables such as 'dTdt'.

Why feature engineering still matters after encoding

- Encoding (Class 14) only makes a variable numeric; it says nothing about whether that variable is clean, trustworthy, or worth keeping.
- Many real datasets carry missing values, duplicates, and inconsistent types that must be resolved before any encoding decision is meaningful.
- Not every available column deserves to be a feature: some are redundant, some are noise, and some are actively misleading.
- The best features are often not present in the raw data at all; they must be constructed from domain knowledge.

Four related but different tasks

Cleaning	Fix missingness, duplicates, invalid values, and basic type errors.
Curation	Organize, document, and validate datasets so they can be reused and trusted.
Selection	Keep the most informative existing variables and drop the rest.
Construction	Build new features from domain logic, time, aggregation, or combinations.

- Start by inventorying missingness, duplicates, and data types before making any modeling decision.
- Missing-value policy should depend on column meaning, not a single global rule.
- Some categorical columns are better served by an explicit “Unknown” category than by imputation.
- Cleaning decisions must be documented, because they change the meaning of the data for everyone downstream.

Cleaning example: typed, column-aware filling

```
titanic = titanic.dropna(subset=["survived"])

for col in ["deck", "embark_town"]:
    titanic[col] = titanic[col].astype("category")
    titanic[col] = titanic[col].cat.add_categories("Unknown")
    titanic[col] = titanic[col].fillna("Unknown")

titanic["age"] = titanic["age"].fillna(titanic["age"].median())
titanic["embarked"] = titanic["embarked"].fillna("S")
```

- From 'notebooks/Class4_DataEngineering.ipynb': three different missingness policies for three different column types in the same dataset.
- Dropping rows, adding an explicit category, and median imputation are all defensible, but only for the right column.

Data curation

- Curation is what makes a cleaned dataset reusable by someone other than the person who cleaned it.
- A data dictionary records what every column means, its unit, and its valid range.
- Versioning the dataset (or at least recording an extraction date) lets later experiments be compared fairly.
- Validation rules (schema checks, range checks) catch upstream problems before they reach modeling.

Repository example

The fraud case's supporting specification in 'documents/' plays the role of a curated data description: it documents the dataset and workflow rather than leaving them implicit in a notebook.

Feature selection: filter, wrapper, embedded

- Filter** Score each feature independently of any model (correlation, chi-squared) and drop low scorers.
- Wrapper** Search over feature subsets by repeatedly training and evaluating a model.
- Embedded** Let the model itself assign importance during training (Lasso coefficients, tree feature importances).

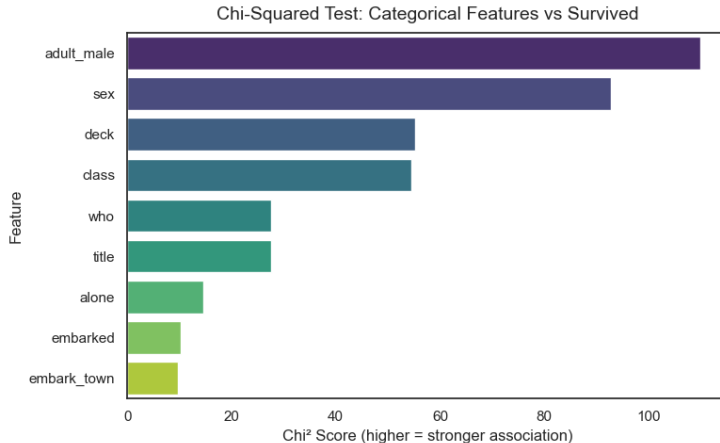
Filter example: chi-squared scores for categorical features

```
encoded = pd.DataFrame()
for col in cat_features.columns:
    le = LabelEncoder()
    encoded[col] = le.fit_transform(cat_features[col].astype(str))

chi_scores, p_values = chi2(encoded, y)
```

- From 'notebooks/Class4_DataEngineering.ipynb': chi-squared is a filter method, computed once, before any model is trained.
- It ranks categorical features by their statistical association with the target, independent of model choice.

Notebook plot: chi-squared scores for categorical features



From 'Class4_DataEngineering.ipynb': not every available categorical feature contributes equally to predicting the outcome.

Selection example: dropping a redundant feature

```
# Remove correlated or redundant features  
# Example: drop 'is_alone' if family_size already encodes similar info  
print(X.shape)  
X_selected = X.drop(columns=["is_alone"])  
print(X_selected.shape)
```

- This is a manual, knowledge-driven form of selection: 'is_alone' is largely redundant once 'family_size' is present.
- Selection does not always need an automated score; sometimes domain reasoning is the fastest filter.

Feature construction from domain knowledge

- The best new features usually come from understanding the problem, not from generic mathematical transforms.
- Aggregation (counts, sums), ratios, and simple domain rules often outperform complex automated feature generation.
- Constructed features should be checked for redundancy against the columns that produced them.
- Students should be able to explain, in one sentence, why a constructed feature should help the model.

Construction example: simple domain logic

```
titanic["family_size"] = titanic["sibsp"] + titanic["parch"] + 1
titanic["is_alone"] = (titanic["family_size"] == 1).astype(int)
titanic["title"] = titanic["who"].map(
    {"man": "Mr", "woman": "Mrs", "child": "Master"}
)
```

- From 'notebooks/Class4_DataEngineering.ipynb': each new column is a short, explainable rule, not a black-box transformation.
- Richer examples for the fraud case: rolling transaction count, deviation from recent spending, merchant risk history.

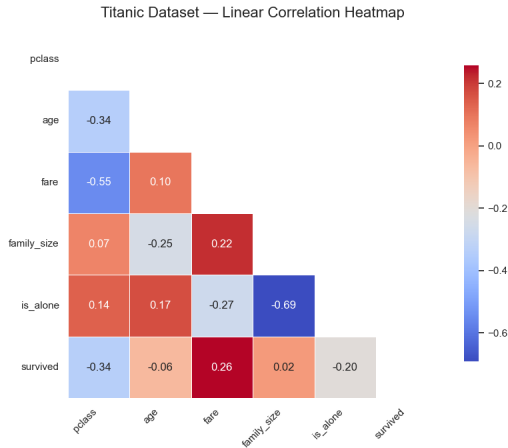
Feature construction for streams and sensors

- Instantaneous sensor values are often less informative than trends and rates of change.
- Window-based features such as rolling means can reveal process stability that a single reading cannot.
- State-aware features distinguish normal variation within a phase from a genuine anomaly.
- Streaming systems must compute these features online, one observation at a time, not only offline in a notebook.

Repository example

'pasteurization/synth_sensors.py' already includes the derived feature 'dTdt', a simple but meaningful process feature built from raw temperature.

Notebook plot: engineered features change the picture



From 'Class4_DataEngineering.ipynb': constructed variables such as 'family_size' and 'title' change how features relate to the target compared to the raw columns alone.

Leakage and availability checks

- Was this feature actually available at prediction time, or does it use information from the future?
- Does it encode a post-decision outcome, such as a manual review result?
- Will the feature be computed the same way in production as it was in the notebook?
- Does the feature depend on unstable joins or upstream systems that may not always be available?

Repo activity

- 'notebooks/Class4_DataEngineering.ipynb' covers cleaning, chi-squared selection, and domain-driven construction end to end.
- 'pasteurization/synth_sensors.py' is the constructed-feature example for time series data ('dTdt').
- 'streamlit/modeling.py' shows why cleaning and construction decisions must be saved alongside the model, not just applied once in a notebook.

Question

'is_alone' was dropped as redundant with 'family_size'. Is that selection decision purely statistical, or does it also depend on domain reasoning?

- This distinguishes filter-style automated selection from the domain judgment that guided construction in the first place.

Papers and materials

Zheng and Casari, Feature Engineering for Machine Learning; Domingos (2012), A Few Useful Things to Know About Machine Learning; scikit-learn pipelines and composite estimators; pandas documentation.

From This Class to the Next

What stays with us

- Cleaning, curation, selection, and construction are four distinct tasks, each with its own failure modes.
- Filter, wrapper, and embedded selection methods answer “which features to keep” in different, complementary ways.
- The strongest features usually come from domain knowledge, not automated transformation alone.

Project use

Project teams should now be able to justify which raw variables were cleaned, kept, dropped, or constructed, and defend each choice on evidence or domain logic.

Next connection

Class 16 turns this feature set into model comparisons, pairs modeling with disciplined experiment tracking and tuning, and asks whether the winning run is actually good for the decision it supports.